

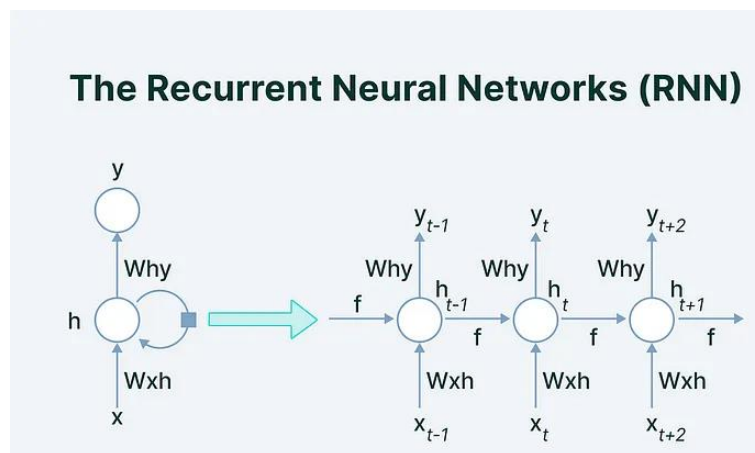
Recurrent Neural Networks (RNNs)

In traditional neural networks, inputs and outputs are treated independently. However, tasks like predicting the next word in a sentence require information from previous words to make accurate predictions. To address this limitation, Recurrent Neural Networks (RNNs) were developed.

Unlike standard neural networks that excel at tasks like image recognition, RNNs boast a unique superpower – memory! This internal memory allows them to analyse sequential data, where the information order is crucial.

RNNs can remember important things about the input they received, which allows them to be very precise in predicting what's coming next.

This Neural Network is called Recurrent because it can repeatedly perform the same task or operation on a sequence of inputs.



All of the inputs and outputs in standard neural networks are independent of one another. However, in some circumstances, such as when predicting the next word of a phrase, the prior words are necessary, and so the previous words must be remembered. As a result, RNN was created, which used a hidden layer to overcome the problem. The most important component of RNN is the hidden state, which remembers specific information about a sequence.

RNNs have a Memory that stores all information about the calculations. They employ the same settings for each input since they produce the same outcome by performing the same task on all inputs or hidden layers.

Recurrent neural networks (RNNs) set themselves apart from other neural networks with their unique capabilities:

- **Internal Memory:** This is the key feature of RNNs. It allows them to remember past inputs and use that context when processing new information.
- **Sequential Data Processing:** Because of their memory, RNNs are exceptional at handling sequential data where the order of elements matters. This makes them ideal

for speech recognition, machine translation, natural language processing (NLP) and text generation.

- **Contextual Understanding:** RNNs can analyse the current input in relation to what they've "seen" before. This contextual understanding is crucial for tasks where meaning depends on prior information.
- **Dynamic Processing:** RNNs can continuously update their internal memory as they process new data, allowing them to adapt to changing patterns within a sequence.

RNN Architecture

RNNs are a type of neural network with hidden states and allow past outputs to be used as inputs. They usually go like this:

Here's a breakdown of its key components:

- **Input Layer:** This layer receives the initial element of the sequence data. For example, in a sentence, it might receive the first word as a vector representation.
- **Hidden Layer:** The heart of the RNN, the hidden layer contains a set of interconnected neurons. Each neuron processes the current input along with the information from the previous hidden layer's state. This "state" captures the network's memory of past inputs, allowing it to understand the current element in context.
- **Activation Function:** This function introduces non-linearity into the network, enabling it to learn complex patterns. It transforms the combined input from the current input layer and the previous hidden layer state before passing it on.
- **Output Layer:** The output layer generates the network's prediction based on the processed information. In a language model, it might predict the next word in the sequence.
- **Recurrent Connection:** A key distinction of RNNs is the recurrent connection within the hidden layer. This connection allows the network to pass the hidden state information (the network's memory) to the next time step. It's like passing a baton in a relay race, carrying information about previous inputs forward

Types of RNN

- **One To One:** There is only one pair here. A one-to-one architecture is used in traditional neural networks.
- **One To Many:** A single input in a one-to-many network might result in numerous outputs. One too many networks are used in music production, for example.
- **Many To One:** A single output combines inputs from distinct time steps in this scenario. Sentiment analysis and emotion identification use such networks, in which a sequence of words determines the class label.

- **Many to Many:** For many to many, there are numerous options. Two inputs yield three outputs. Machine translation systems, such as English to French or vice versa translation systems, use many-to-many networks.

Forward Pass:

Hidden State Update:

At each time step t , the RNN updates its hidden state h_t based on the previous hidden state h_{t-1} and the current input x_t .

$$h_t = \tanh(W_{hh} * h_{t-1} + W_{xh} * x_t + b_h)$$

Where:

- h_t is the hidden state at time t .
- W_{hh} is the weight matrix for the hidden-to-hidden connection.
- W_{xh} is the weight matrix for the input-to-hidden connection.
- x_t is the input at time t (typically a one-hot encoded vector or an embedding).
- b_h is the bias term for the hidden layer.
- $\tanh$ is the activation function used to compute the hidden state (other activations like ReLU or sigmoid can also be used).

Output Layer:

The output of the RNN at time step t is based on the hidden state h_t and is passed through a linear transformation and a softmax activation (for classification tasks like predicting the next word).

$$y_t = W_{hy} * h_t + b_y$$

Where:

- y_t is the raw output (logits) at time t .
- W_{hy} is the weight matrix for the hidden-to-output connection.
- b_y is the bias term for the output layer.

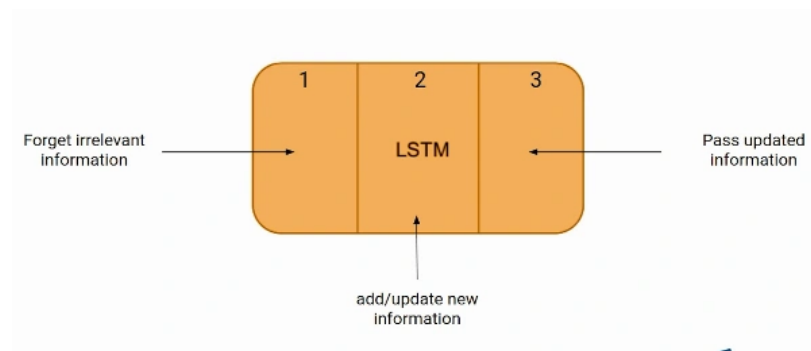
Long Short-Term Memory

LSTM is a type of RNN with higher memory power to remember the outputs of each node for a more extended period to produce the outcome for the next node efficiently.

LSTM networks combat the RNN's vanishing gradients or long-term dependence issue.

Gradient vanishing refers to the loss of information in a neural network as connections recur over a longer period.

The LSTM network architecture consists of three parts, as shown in the image below, and each part performs an individual function.



The first part chooses whether the information coming from the previous timestamp is to be remembered or is irrelevant and can be forgotten.

In the second part, the cell tries to learn new information from the input to this cell.

At last, in the third part, the cell passes the updated information from the current timestamp to the next timestamp. This one cycle of LSTM is considered a single-time step.

These three parts of an LSTM unit are known as gates.

They control the flow of information in and out of the memory cell or lstm cell.

The first gate is called Forget gate, the second gate is known as the Input gate, and the last one is the Output gate.

An LSTM unit that consists of these three gates and a memory cell or lstm cell can be considered as a layer of neurons in traditional feedforward neural network, with each neuron having a hidden layer and a current state.

NEXTWORD PREDICTION

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
```

```
from tensorflow.keras.utils import to_categorical
```

- imports the `Tokenizer` class from TensorFlow's Keras preprocessing module.

This is a **powerful and commonly used utility for text preprocessing in NLP projects**.

Tokenizer is a class that:

- Converts raw text (sentences) into sequences of integers.
- Creates a word-to-index mapping (vocabulary).
- Helps you easily prepare input data for training models like RNNs, LSTMs, and Transformers.

Key Features of Tokenizer

Feature	Description
<code>.fit_on_texts()</code>	Analyzes a corpus and builds the word index (vocab).
<code>.texts_to_sequences()</code>	Converts words in a sentence to a list of integers (based on word index).
<code>.sequences_to_texts()</code>	Converts sequences of integers back into words.
<code>.word_index</code>	Dictionary mapping words to integer IDs.
<code>.index_word</code>	Reverse of <code>word_index</code> – maps integers back to words.

- imports the `pad_sequences` utility function from TensorFlow's Keras module.

This is **used to ensure that all input sequences in your model have the same length** — which is **mandatory for deep learning models** in Keras.

Neural networks expect input data to be of uniform shape. But sentences or sequences often have different lengths. `pad_sequences` helps by:

- Padding **shorter sequences** with zeros (or any value) so they match the longest one.
- Optionally **truncating longer sequences**.
- Making the data suitable to feed into models like RNNs or LSTMs.

Example:

```
sequences = [  
    [1, 2, 3],  
    [4, 5],  
    [6] ]
```

`pad_sequences` convert it to

```
[[1 2 3]  
 [0 4 5]  
 [0 0 6]]
```

```
pad_sequences(  
    sequences,  
    maxlen=None,  
    dtype='int32',  
    padding='pre',  
    truncating='pre',  
    value=0  
)
```

Parameter	Description
<code>`maxlen`</code>	Maximum length of all sequences (you can <code>set</code> it manually).
<code>`padding`</code>	<code>`pre`</code> (default) or <code>`post`</code> . Where to add the padding.
<code>`truncating`</code>	<code>`pre`</code> or <code>`post`</code> . Where to truncate longer sequences.
<code>`value`</code>	Padding value (default is <code>`0`</code>).

- **What is Embedding ?**

The Embedding layer maps integer-encoded words (like 1, 2, 3) into dense vector representations (real-valued vectors), which are learned during training.

It's commonly used as the first layer in models that deal with text input — like RNNs,

Words are not inherently numeric, so we first convert them to integers using something like a Tokenizer. But we can't directly feed integers like 4 or 27 into a neural network — it won't understand the relationship between words like "love" and "like".

The Embedding layer solves this by learning **low-dimensional continuous vector representations** of words. These vectors capture meaning and relationships between words.

• What is to_categorical?

to_categorical converts a class label (integer like 0, 1, 2, ...) into a **one-hot encoded vector**.

This is essential when:

- You're dealing with classification problems, and
- The model output layer uses softmax activation with categorical crossentropy loss.

```
corpus = [
    "i love machine learning",
    "i love deep learning",
    "i love natural language processing",
    "deep learning is fun",
    "natural language processing is a field of ai",
    "i enjoy learning new things"
]
```

```
# Tokenize the text
tokenizer = Tokenizer()
tokenizer.fit_on_texts(corpus)

total_words = len(tokenizer.word_index) + 1 # add 1 for padding
```

```
# Create input sequences using n-grams
input_sequences = []

for line in corpus:
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):

        n_gram_seq = token_list[:i+1]
        input_sequences.append(n_gram_seq)

# Pad sequences
max_seq_len = max([len(x) for x in input_sequences])
input_sequences = pad_sequences(input_sequences, maxlen=max_seq_len,
padding='pre')
```

```
X = input_sequences[:, :-1]
y = input_sequences[:, -1]

# One-hot encode labels
y = to_categorical(y, num_classes=total_words)
y
```

```
model = Sequential()
model.add(Embedding(input_dim=total_words, output_dim=64,
input_length=max_seq_len-1))
model.add(SimpleRNN(100))
model.add(Dense(total_words, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
model.summary()
```

```
model.fit(X, y, epochs=300, verbose=1)
```

verbose=0 **No output** during training (silent mode)

verbose=1 **Progress bar** with epoch metrics (default)

verbose=2 **One line per epoch** (no progress bar)

```
def predict_next_word(model, tokenizer, text_seed, max_seq_len):
    token_list = tokenizer.texts_to_sequences([text_seed])[0]
    token_list = pad_sequences([token_list], maxlen=max_seq_len - 1,
padding='pre')
    predicted = model.predict(token_list, verbose=0)
    predicted_word_index = np.argmax(predicted)

    for word, index in tokenizer.word_index.items():
        if index == predicted_word_index:
            return word
    return ""
```

```
seed_text = "i love"
next_word = predict_next_word(model, tokenizer, seed_text, max_seq_len)
print(f"Seed text: {seed_text}")
print(f"Predicted next word: {next_word}")
```